

stride_view

Document #: P1899R2
Date: 2021-12-20
Project: Programming Language C++
Audience: SG9
LEWG
Reply-to: Christopher Di Bella
<cjdb.ns@gmail.com>
Tim Song
<t.canens.cpp@gmail.com>

Contents

- [1 Abstract](#)
- [2 Revision history](#)
 - [2.1 R2](#)
 - [2.2 R1](#)
 - [2.3 R0](#)
- [3 Motivation](#)
 - [3.1 Risk of not having `stride\_view`](#)
- [4 Implementation experience](#)
- [5 Design notes](#)
 - [5.1 Preconditions](#)
 - [5.2 Complex iteration model](#)
 - [5.3 Properties](#)
- [6 Proposed wording](#)
 - [6.1 Addition to `<ranges>`](#)
 - [6.2 `stride`](#)
 - [24.7.? Stride view \[`range.stride`\]](#)
 - [24.7.?.1 Overview \[`range.stride.overview`\]](#)
 - [24.7.?.2 Class template `stride\_view` \[`range.stride.view`\]](#)
 - [24.7.?.3 Class template `stride\_view::iterator` \[`range.stride.iterator`\]](#)
 - [6.3 Feature-test macro](#)
- [7 Acknowledgements](#)
- [8 References](#)

§ 1 Abstract

The ability to use algorithms over an evenly-spaced subset of a range has been missed in the STL for a quarter of a century. Given that there's no way to compose a strided range adaptor in C++20, this should be adopted for C++23.

§ 2 Revision history

§ 2.1 R2

- Rewrite wording to match chunk from [\[P2442R1\]](#).

§ 2.2 R1

- PDF -> HTML.
- Adds a section discussing the design.
- Adds feature-test macro.
- Cleans up some stuff that was ported in from the implementation by mistake.
- Adds `iterator_concept`, and corrects `iterator_category` so it can't be contiguous.
- Fixes calls to *compute-distance* so they pass in size of underlying range instead of themselves.
- Adds precondition to ensure stride is positive.
- Makes multi-arg constructors non-explicit.

§ 2.3 R0

Initial revision.

§ 3 Motivation

The ability to use algorithms over an evenly-spaced subset of a range has been missed in the STL for a quarter of a century. This is, in part, due to the complexity required to use an iterator that can safely describe such a range. It also means that the following examples cannot be transformed from raw loops into algorithms, due to a lacking iterator.

```
namespace stdr = std::ranges;
namespace stdv = std::views;

for (auto i = 0; i < ssize(v); i += 2) {
    v[i] = 42; // fill
}

for (auto i = 0; i < ssize(v); i += 3) {
    v[i] = f(); // transform
}

for (auto i = 0; i < ssize(v); i += 3) {
    for (auto j = i; j < ssize(v); j += 3) {
        if (v[j] < v[i]) {
            stdr::swap(v[i], v[j]); // selection sort, but hopefully the idea is conveyed
        }
    }
}
```

```
}  
}
```

Boost.Range 2.0 introduced a range adaptor called `strided`, and range-v3's equivalent is `stride_view`, both of which make striding far easier than when using iterators:

```
stdr::fill(v | stdv::stride(2), 42);  
  
auto strided_v = v | stdv::stride(3);  
stdr::transform(strided_v, stdr::begin(strided_v) f);  
  
stdr::stable_sort(strided_v); // order restored!
```

Given that there's no way to compose a strided range adaptor in C++20, this should be one of the earliest range adaptors put into C++23.

§ 3.1 Risk of not having `stride_view`

Although it isn't possible to compose `stride_view` in C++20, someone inexperienced with the ranges design space might mistake `filter_view` as a suitable way to “compose” `stride_view`:

```
auto bad_stride = [](auto const step) {  
    return views::filter([n = 0, step](auto&&) mutable {  
        return n++ % step == 0;  
    });  
};
```

This implementation is broken for two reasons:

1. `filter_view` expects a predicate as its input, but the lambda we have provided does not model predicate (a call to invoke on a predicate mustn't modify the function object, yet we clearly are).
2. The lambda provided doesn't account for moving backward, so despite *satisfying* `bidirectional_iterator`, it does not model the concept, thus rendering any program containing it ill-formed, with no diagnostic being required.

For these reasons, the author regrets not proposing this in the C++20 design space.

§ 4 Implementation experience

Both Boost.Range 2.0 and range-v3 are popular ranges libraries that support a striding range adaptor. The proposed wording has mostly been implemented in `cmcstl2` and in a CppCon main session.

§ 5 Design notes

§ 5.1 Preconditions

Boost.Range 2.0's `strided` has a precondition that `0 <= n`, but this isn't strong enough: we need `n` to be *positive*.

The stride needs to be positive since a negative stride doesn't really make sense, and a semantic requirement of `std::weakly_increamentable` (23.3.4.4 [\[iterator.concept.winc\]](#)) is that incrementing actually moves the iterator to the next element: this means a zero-stride isn't allowed either.

LEWG unanimously agreed that this was the correct decision in Prague.

§ 5.2 Complex iteration model

A simple implementation of `stride_view` would be similar to what's in Boost.Range 2.0: a single-pass range adaptor. With some effort, we can go all the way to a random-access range adaptor, which is what this section mainly covers.

A naive random-access range adaptor would be implemented by simply moving the iterator forward or backward by `n` positions (where `n` is the stride length). While this produces a correct iterator when moving forward, its `operator--` will be incorrect whenever `n` doesn't evenly divide the underlying range's length. For example:

```
auto x = std::vector{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};

// prints 0 3 6 9
stdr::copy(stdv::stride(x, 3), std::ostream_iterator<int>(std::cout, " "));

// prints 9 6 3 0
stdr::copy(stdv::stride(x, 3) | stdv::reverse, std::ostream_iterator<int>(std::cout, "
"));

auto y = std::vector{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// prints 0 3 6 9
stdr::copy(stdv::stride(y, 3), std::ostream_iterator<int>(std::cout, " "));

// prints 8 5 2: not the same range in reverse!?
stdr::copy(stdv::stride(y, 3) | stdv::reverse, std::ostream_iterator<int>(std::cout, "
"));
```

The problem here is that going from the one-before-past-the-end iterator to the past-the-end iterator may involve iterating fewer than stride steps, so to correctly iterate backwards, we need to know that distance.

This is the same problem as `chunk` ([\[P2442R1\]](#)) and can be solved in the same way. After all, `stride(n)` is basically a more efficient version of `chunk(n) | transform(ranges::front)` (if we actually had a `ranges::front`).

§ 5.3 Properties

`stride_view` is borrowed if the underlying view is. It is a common range if the underlying range is common and either sized or non-bidirectional.

§ 6 Proposed wording

§ 6.1 Addition to `<ranges>`

Add the following to 24.2 [\[ranges.syn\]](#), header `<ranges>` synopsis:

```
// [...]
namespace std::ranges {
    // [...]

    // [range.stride], stride view
    template<input_range V>
        requires view<V>
        class stride_view;

    template<class V>
        inline constexpr bool enable_borrowed_range<stride_view<V>> =
            enable_borrowed_range<V>;

    namespace views { inline constexpr unspecified stride = unspecified; }

    // [...]
}
```

§ 6.2 stride

Add the following subclause to 24.7 [\[range.adaptors\]](#).

[Editor's note: This wording assumes the exposition-only *div-ceil* function template introduced in [\[P2442R1\]](#).]

§ 24.7.? Stride view `[range.stride]`

§ 24.7.?.1 Overview `[range.stride.overview]`

- 1 stride_view presents a view of an underlying sequence, advancing over n elements at a time, as opposed to the usual single-step succession.
- 2 The name `views::stride` denotes a range adaptor object 24.7.2 [\[range.adaptor.object\]](#). Given subexpressions E and N, the expression `views::stride(E, N)` is expression-equivalent to `stride_view(E, N)`.
- 3 [Example:

```
auto input = views::iota(0, 12) | views::stride(3);
ranges::copy(input, ostream_iterator<int>(cout, " ")); // prints 0 3 6 9
ranges::copy(input | views::reverse, ostream_iterator<int>(cout, " ")); // prints 9 6
                                3 0
```

— end example]

§ 24.7.?.2 Class template `stride_view` [`range.stride.view`]

```
namespace std::ranges {
    template<input_range V>
        requires view<V>
    class stride_view : public view_interface<stride_view<V>> {
        V base_ = V(); // exposition only
        range_difference_t<V> stride_ = 1; // exposition only
        template<bool> class iterator; // exposition only
    public:
        stride_view() requires default_initializable<V> = default;
        constexpr explicit stride_view(V base, range_difference_t<V> stride);

        constexpr V base() const& requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr range_difference_t<V> stride() const noexcept;

        constexpr auto begin() requires (!simple-view<V>) {
            return iterator<false>(this, ranges::begin(base_));
        }

        constexpr auto begin() const requires range<const V> {
            return iterator<true>(this, ranges::begin(base_));
        }

        constexpr auto end() requires (!simple-view<V>) {
            if constexpr (common_range<V> && sized_range<V> && forward_range<V>) {
                auto missing = (stride_ - ranges::distance(base_) % stride_) % stride_;
                return iterator<false>(this, ranges::end(base_), missing);
            }
            else if constexpr (common_range<V> && !bidirectional_range<V>) {
                return iterator<false>(this, ranges::end(base_));
            }
            else {
                return default_sentinel;
            }
        }

        constexpr auto end() const requires range<const V> {
            if constexpr (common_range<const V> && sized_range<const V> &&
                forward_range<const V>) {
                auto missing = (stride_ - ranges::distance(base_) % stride_) % stride_;
                return iterator<true>(this, ranges::end(base_), missing);
            }
            else if constexpr (common_range<const V> && !bidirectional_range<const V>) {
                return iterator<true>(this, ranges::end(base_));
            }
            else {
                return default_sentinel;
            }
        }

        constexpr auto size() requires sized_range<V>;
        constexpr auto size() const requires sized_range<const V>;
    };
}
```

```
};

template<class R>
    stride_view(R&&, range_difference_t<R>) -> stride_view<views::all_t<R>>;
}
```

[Drafting note: `end()` cannot compute missing for input-only ranges because `ranges::size` (and `ranges::distance` by extension) is not required to be valid after `ranges::begin` is called, but `end()` must remain callable.]

```
constexpr stride_view(V base, range_difference_t<V> stride);
```

- 1 *Preconditions:* `stride > 0` is true.
- 2 *Effects:* Initializes `base_` with `std::move(base)` and `stride_` with `stride`.

```
constexpr range_difference_t<V> stride() const;
```

- 3 *Returns:* `stride_`.

```
constexpr auto size() requires sized_range<V>;
constexpr auto size() const requires sized_range<const V>;
```

- 4 *Effects:* Equivalent to:

```
return to-unsigned-like(div-ceil(ranges::distance(base_),
    stride_));
```

§ 24.7.3 Class template `stride_view::iterator` [range.stride.iterator]

```
namespace std::ranges {
    template<input_range V>
        requires view<V>
        template<bool Const>
        class stride_view<V>::iterator {
            using Parent = maybe_const<Const, chunk_view>;           // exposition only
            using Base = maybe_const<Const, V>;                     // exposition only

            iterator_t<Base> current_ = iterator_t<Base>();          // exposition only
            sentinel_t<Base> end_ = sentinel_t<Base>();              // exposition only
            range_difference_t<Base> stride_ = 0;                    // exposition only
            range_difference_t<Base> missing_ = 0;                   // exposition only

            constexpr iterator(Parent* parent, iterator_t<Base> current, // exposition only
                               range_difference_t<Base> missing = 0);

        public:
            using difference_type = range_difference_t<Base>;
            using value_type = range_value_t<Base>;
            using iterator_concept = see below;
            using iterator_category = see below; // not always present

            iterator() requires default_initializable<iterator_t<Base>>= default;

            constexpr iterator(iterator<!Const> other)
```

```

    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>
               && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

constexpr iterator_t<Base> base() &&;
constexpr const iterator_t<Base>& base() const & noexcept;

constexpr decltype(auto) operator*() const { return *current_; }

constexpr iterator& operator++();

constexpr void operator++(int);
constexpr iterator operator++(int) requires forward_range<Base>;

constexpr iterator& operator--() requires bidirectional_range<Base>;
constexpr iterator operator--(int) requires bidirectional_range<Base>;

constexpr iterator& operator+=(difference_type n) requires
    random_access_range<Base>;
constexpr iterator& operator-=(difference_type n) requires
    random_access_range<Base>;

constexpr decltype(auto) operator[](difference_type n) const
    requires random_access_range<Base>
{ return *(*this + n); }

friend constexpr bool operator==(const iterator& x, default_sentinel);

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;

friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> && three_way_comparable<iterator_t<Base>>;

friend constexpr iterator& operator+(const iterator& x, difference_type n)
    requires random_access_range<Base>;
friend constexpr iterator& operator+(difference_type n, const iterator& x)
    requires random_access_range<Base>;
friend constexpr iterator& operator-(const iterator& x, difference_type n)
    requires random_access_range<Base>;
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

friend constexpr difference_type operator-(default_sentinel_t y, const iterator&
    x)

```



```

    requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;
    friend constexpr difference_type operator-(const iterator& x, default_sentinel_t
        y)
        requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

    friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
        noexcept(noexcept(ranges::iter_move(i.current_)));

    friend constexpr void iter_swap(const iterator& x, const iterator& y)
        noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
        requires indirectly_swappable<iterator_t<Base>>;
};
}

```

1 *iterator::iterator_concept* is defined as follows:

- .1) — If *Base* models *random_access_range*, then *iterator_concept* denotes *random_access_iterator_tag*.
- .2) — Otherwise, if *Base* models *bidirectional_range*, then *iterator_concept* denotes *bidirectional_iterator_tag*.
- .3) — Otherwise, if *Base* models *forward_range*, then *iterator_concept* denotes *forward_iterator_tag*.
- .4) — Otherwise, *iterator_concept* denotes *input_iterator_tag*.

2 The member *typedef-name* *iterator_category* is defined if and only if *Base* models *forward_range*. In that case, *iterator::iterator_category* is defined as follows:

- .1) — Let *C* denote the type *iterator_traits<iterator_t<Base>>::iterator_category*.
- .2) — If *C* models *derived_from<random_access_iterator_tag>*, then *iterator_category* denotes *random_access_iterator_tag*.
- .3) — Otherwise, *iterator_category* denotes *C*.

```

constexpr iterator(Parent* parent, iterator_t<Base> current,
    range_difference_t<Base> missing = 0);

```

- 3 *Effects*: Initializes *current_* with `std::move(current)`, *end_* with `ranges::end(parent->base_)`, *stride_* with `parent->stride_`, and *missing_* with *missing*.

```

constexpr iterator(iterator<!Const> i)
    requires Const && convertible_to<iterator_t<V>, iterator_t<Base>>
        && convertible_to<sentinel_t<V>, sentinel_t<Base>>;

```

- 4 *Effects*: Initializes *current_* with `std::move(i.current_)`, *end_* with `std::move(i.end_)`, *stride_* with `i.stride_`, and *missing_* with `i.missing_`.

```

constexpr iterator_t<Base> base() &&;

```

- 5 *Returns*: `std::move(current_)`.

```
constexpr const iterator_t<Base>& base() const & noexcept;
```

6 *Returns:* `current_`.

```
constexpr iterator& operator++();
```

7 *Preconditions:* `current_ != end_` is `true`.

8 *Effects:* Equivalent to:

```
missing_ = ranges::advance(current_, stride_, end_);  
return *this;
```

```
constexpr void operator++(int);
```

9 *Effects:* Equivalent to: `++*this`;

```
constexpr iterator operator++(int) requires forward_range<Base>;
```

10 *Effects:* Equivalent to:

```
auto tmp = *this;  
++*this;  
return tmp;
```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

11 *Effects:* Equivalent to:

```
ranges::advance(current_, missing_ - stride_);  
missing_ = 0;  
return *this;
```

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

```
auto tmp = *this;  
--*this;  
return tmp;
```

```
constexpr iterator& operator+=(difference_type n) requires random_access_range<Base>;
```

12 *Preconditions:* If `n` is positive, `ranges::distance(current_, end_) > stride_ * (n - 1)` is `true`. [*Note 1:* If `n` is negative, the *Effects:* paragraph implies a precondition. — *end note*]

13 *Effects:* Equivalent to:

```
if (n > 0) {  
    missing_ = ranges::advance(current_, stride_ * n, end_);  
}  
else if (n < 0) {  
    ranges::advance(current_, stride_ * n + missing_);  
    missing_ = 0;  
}  
return *this;
```

```
constexpr iterator& operator--(difference_type x)
    requires random_access_range<Base>;
```

14 *Effects:* Equivalent to: `return *this += -x;`

```
friend constexpr bool operator==(const iterator& x, default_sentinel);
```

15 *Returns:* `x.current_ == x.end_;`

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;
```

16 *Returns:* `x.current_ == y.current_.`

```
friend constexpr bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

17 *Returns:* `x.current_ < y.current_.`

```
friend constexpr bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

18 *Effects:* Equivalent to: `return y < x;`

```
friend constexpr bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

19 *Effects:* Equivalent to: `return !(y < x);`

```
friend constexpr bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;
```

20 *Effects:* Equivalent to: `return !(x < y);`

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
           three_way_comparable<iterator_t<Base>>;
```

21 *Returns:* `x.current_ <=> y.current_.`

```
friend constexpr iterator operator+(const iterator& i, difference_type n)
    requires random_access_range<Base>;
```

```
friend constexpr iterator operator+(difference_type n, const iterator& i)
    requires random_access_range<Base>;
```

22 *Effects:* Equivalent to:

```
    auto r = i;
    r += n;
    return r;
```

```
friend constexpr iterator operator-(const iterator& i, difference_type n)
    requires random_access_range<Base>;
```

23 *Effects:* Equivalent to:

```

auto r = i;
r -= n;
return r;

```

```

friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires sized_sentinel_for<iterator_t<Base>, iterator_t<Base>>;

```

24 *Returns:* Let N be $x.current_ - y.current_.$

(24.1) — If *Base* models *forward_range*, $(N + x.missing_ - y.missing_)/x.stride_.$

(24.2) — Otherwise, if N is negative, $-div_ceil(-N, x.stride_).$

(24.3) — Otherwise, $div_ceil(N, x.stride_).$

[Drafting note: When *Base* is input-only, the value of *missing_* is unreliable.]

```

friend constexpr difference_type operator-(default_sentinel_t y, const iterator& x)
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

```

25 *Returns:* $div_ceil(x.end_ - x.current_ , x.stride_).$

```

friend constexpr difference_type operator-(const iterator& x, default_sentinel_t y)
requires sized_sentinel_for<sentinel_t<Base>, iterator_t<Base>>;

```

26 *Effects:* Equivalent to: `return -(y - x);`

```

friend constexpr range_rvalue_reference_t<Base> iter_move(const iterator& i)
noexcept(noexcept(ranges::iter_move(i.current_)));

```

27 *Effects:* Equivalent to: `return ranges::iter_move(i.current_);`

```

friend constexpr void iter_swap(const iterator& x, const iterator& y)
noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
requires indirectly_swappable<iterator_t<Base>>;

```

28 *Effects:* Equivalent to: `ranges::iter_swap(x.current_, y.current_);`

§ 6.3 Feature-test macro

Add the following macro definition to 17.3.2 [\[version.syn\]](#), header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```

#define __cpp_lib_ranges_stride 20XXXXL // also in <ranges>

```

§ 7 Acknowledgements

The author would like to thank Tristan Brindle for providing editorial commentary on P1899, and also those who reviewed material for, or attended the aforementioned CppCon session or post-conference class, for their input on the design of the proposed `stride_view`.

§ 8 References

[P2442R1] Tim Song. 2021. Windowing range adaptors: `views::chunk` and `views::slide`.
<https://wg21.link/P2442R1>